



**UNIVERSITY OF LAY ADVENTISTS OF KIGALI
(UNILAK)**

PO Box: 6392 Kigali-Rwanda
Phone: +250731743430/+250731743431

Website: www.unilak.ac.rw
E-mail: info@unilak.ac.rw

FACULTY: CIS

DEPARTMENT: SOFTWARE ENGINEERING

ACADEMIC YEAR: 2025-2026

YEAR OF STUDY: II

MODULE TITLE: SOFTWARE TESTING & QUALITY ASSURANCE

SESSION: EVENING

MODULE WEIGHT: 10 CREDITS

DURATION: 4 Days

CATEGORY OF ASSESSMENT: Final Exam

DATE: January 21 , 2026

MAXIMUM: 40 MARKS

Lecturer: Shema Lucien

CAMPUS: Kigali

INSTRUCTIONS

- 1. This FINAL EXAM is a Practical Assignment (Take Home)**
- 2. Duration: 4 days from the date of issue**
- 3. All components listed in the requirements section must be completed**
- 4. Submit all deliverables before the deadline to avoid penalties**

Full-Stack Todo Application

Overview

Extend your existing CRUD Todo application with authentication, authorization, user roles, and advanced features to create a production-ready application.

Objectives

- Implement JWT-based authentication and authorization
- Design and implement role-based access control (RBAC)
- Apply security best practices
- Build advanced API features
- Create comprehensive testing suites
- Deploy a full-stack application (optional)

Technical Stack

- **Backend:** Node.js, Express.js
- **Database:** MongoDB with Mongoose
- **Authentication:** JWT, bcrypt
- **Testing:** Jest, Mocha Chai, Supertest
- **Frontend:** React/Js Next Js (or your preferred framework)
- **Deployment:** Your choice (Heroku, Netlify, Vercel, etc.)

Part 1: Authentication & Authorization

1.1 User Management System

- **User Registration**
 - Email validation and uniqueness
 - Password strength requirements (min 8 chars, 1 uppercase, 1 number, 1 special char)
 - Password hashing with bcrypt (salt rounds: 12)
 - Email verification (bonus: implement email sending)
- **User Login**
 - JWT token generation (expires in 24h)
 - Refresh token mechanism
 - Account lockout after 3 failed attempts
- **User Profile**

- View and update profile information
- Change password functionality
- Delete account option

1.2 Middleware Implementation

- **Authentication Middleware:** Verify JWT tokens
- **Authorization Middleware:** Check user permissions
- **Error Handling Middleware:** Centralized error responses
- **Request Validation Middleware:** Validate request bodies

Part 2: Role-Based Access Control

2.1 User Roles

Implement three user roles with different permissions:

- **Admin**
 - Full system access
 - Manage all users and todos
 - View system analytics
 - Export data
- **Manager**
 - Manage team members' todos
 - View team analytics
 - Assign todos to team members
- **User (Default)**
 - CRUD operations on own todos
 - View own profile and statistics

2.2 Permission System

- Create permission-based middleware
- Implement role hierarchy
- Dynamic permission checking
- Route protection based on roles

Part 3: Advanced Todo Features

3.1 Enhanced Todo Model

Extend the Todo schema with:

```
{  
  title: String (required),  
  description: String,  
  status: Enum ['pending', 'in-progress', 'completed'],  
  priority: Enum ['low', 'medium', 'high'],  
  dueDate: Date,  
  assignedTo: ObjectId (ref: User),  
  assignedBy: ObjectId (ref: User),  
  category: String,  
  tags: [String],  
  createdBy: ObjectId (ref: User),  
  createdAt: Date,  
  updatedAt: Date,  
  completedAt: Date  
}
```

3.2 Advanced CRUD Operations

- **Filtering:** By status, priority, category, assigned user, date range
- **Sorting:** By due date, priority, creation date, status
- **Pagination:** Implement cursor-based pagination
- **Search:** Full-text search across title and description
- **Bulk Operations:** Update multiple todos, bulk delete

3.3 Todo Assignment System

- Managers can assign todos to users
- Users receive notifications for new assignments
- Track assignment history
- Auto-completion tracking

Part 4: Testing & Quality Assurance

4.1 Backend Testing

- **Unit Tests:** All service functions
- **Integration Tests:** API endpoints
- **Authentication Tests:** Login, registration, JWT validation
- **Authorization Tests:** Role-based access
- **Database Tests:** Model validations and relationships

4.2 Testing Requirements

- Minimum 80% code coverage
- Test database setup/teardown
- Mock external dependencies
- Testing error scenarios
- Performance testing for key endpoints

Part 5: Frontend Integration (Optional)

5.1 User Interface

- **Authentication Pages:** Login, Register, Profile
- **Dashboard:** Role-specific dashboards
- **Todo Management:** Advanced todo interface with filtering/sorting
- **Admin Panel:** User management interface (Admin only)

5.2 State Management

- Implement proper state management (Context API/Redux)
- Handle authentication state globally
- Implement loading states and error handling
- Real-time updates (bonus: WebSocket integration)

Submission Requirements

1. Code Repository

- Clean, well-documented Git repository
- Proper commit history with meaningful messages
- README with setup instructions
- Environment configuration examples

2. Documentation

- API documentation (Swagger/Postman collection)
- Database schema documentation
- Deployment guide
- User manual

3. Live Deployment (Optional)

- Deployed backend API
- Deployed frontend application
- Environment variables properly configured
- HTTPS enabled

Evaluation Criteria

Technical Implementation

- **Code Quality:** Clean, readable, and maintainable code
- **Architecture:** Proper separation of concerns, modular design
- **Security:** Proper authentication, authorization, data validation
- **Error Handling:** Comprehensive error handling and logging
- **Performance:** Optimized queries, proper indexing

Features & Functionality

- **Completeness:** All required features implemented
- **User Experience:** Intuitive and responsive interface
- **Advanced Features:** Creative additional features
- **Integration:** Smooth frontend-backend integration

Testing & Documentation

- **Test Coverage:** Comprehensive test suite

- **Documentation Quality:** Clear and complete documentation
- **Code Comments:** Well-commented code where necessary

Deployment & Best Practices

- **Successful Deployment:** Working live application
- **Environment Management:** Proper configuration management
- **Git Practices:** Clean commit history and branching

Resources & References

- [JWT Best Practices](#)
- [Node.js Security Checklist](#)
- [MongoDB Security](#)
- [Express.js Best Practices](#)

Getting Started

1. Create a new branch from your existing todo project
2. Set up the new project structure
3. Install additional dependencies (jsonwebtoken, bcrypt, etc.)
4. Begin with Part 1 implementation
5. Test each part thoroughly before proceeding

SUBMISSION GUIDELINES

Submission Requirements:

- Submit your GitHub repository link via email to luciens@alumni.cmu.edu
- Use the subject line format: RegNumber_Final_Exam
- Set your repository to private
- Add shemalucien as a collaborator
- Include a README file with setup instructions
- Late submissions will not be accepted

Important Dates:

- **Code submission deadline: Tuesday, January 20, 2026 at 11:59 PM**
- **Presentation date: Wednesday, January 21, 2026 from 5:30 PM to 9:30 PM**
 - For students who meet finance requirements

Good luck!